



Performance Characterization of Lattice-based Cryptography Workloads (Poster Slides)

ISPASS 2020

Deepika Natarajan, Prof. Ronald G. Dreslinski

University of Michigan, Ann Arbor, MI

{dnataraj, rdreslin}@umich.edu

Challenge: How can quantum-resistant crypto be performant?

What is Lattice-based Cryptography (LBC)?

- LBC refers to a set of cryptographic techniques that derive their security from the presumed hardness of mathematical problems involving lattices

Why is LBC important?

- Quantum computing will render modern public-key cryptosystems (e.g. RSA, ElGamal) insecure
- LBC schemes have emerged as leading contenders to replace quantum-insecure schemes

This work:

- Flexible domain-specific LBC ASICs are a promising yet relatively unexplored means of accelerating LBC schemes
- We investigate bottlenecks of LBC schemes and amenability of LBC to acceleration techniques

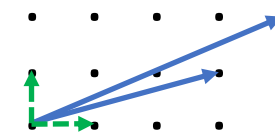


Figure 1. Example of a 2D lattice and a set of basis vectors (blue, solid) for the lattice. A well-known lattice problem involves finding a short vector (green, dashed) in a lattice given a lattice basis. Since this problem is thought to be difficult to solve in polynomial time (given a particular basis), cryptographic constructions based on this problem are thought to be secure.

A Closer Look at LBC Schemes

- We focused on two LBC schemes described in prior work¹: the Compact Public-Key Cryptosystem (CPKC) and the Dual-Style Cryptosystem (DSC)
- Elements in these schemes are polynomials with degree less than an integer m and coefficients in the range $[0, q)$ for an integer q . All operations take place in this “polynomial ring”
- The “Toolkit for Ring-LWE”¹ describes linear transformations on these elements to efficiently perform operations required for encryption and decryption
- Block diagrams (see: Figure 2) show the commonalities between the cryptosystems
- **Example (our focus):** NTT is a common algorithmic means of accelerating the bottleneck of polynomial multiplication

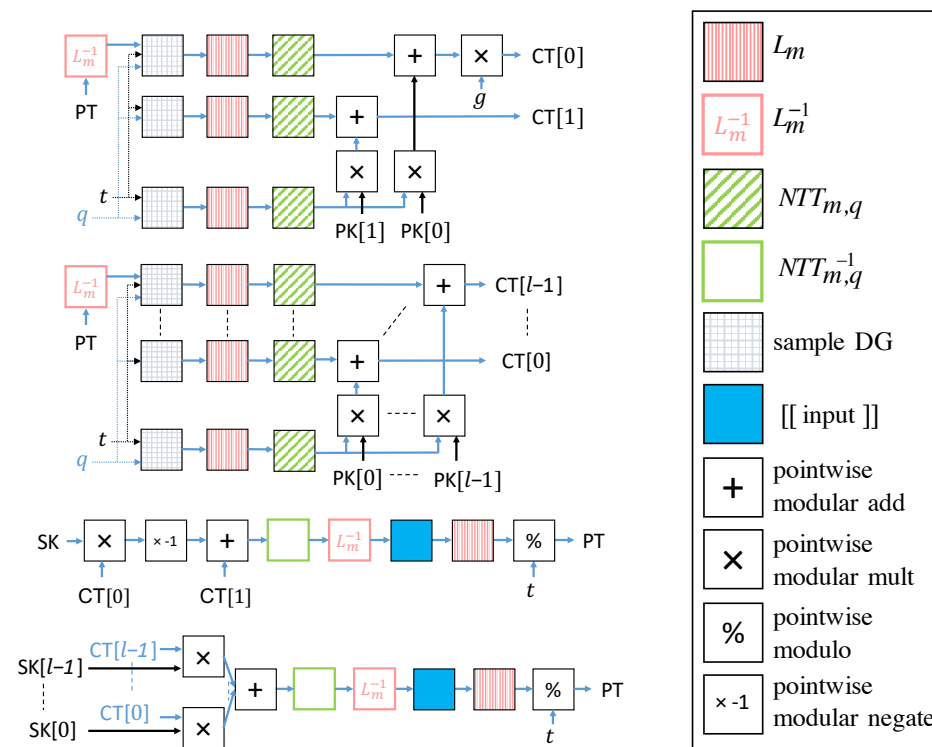


Figure 2. Top to bottom: Block diagrams for CPKC and DSC Encryption and CPKC and DSC Decryption submodules. All arithmetic operations occur modulo a modulus q . Dashes indicate block repetition when $l > 2$. SK, PK, PT, and CT denote the secret key, public key, plaintext, and ciphertext, respectively. “%” denotes the modulo operation and “[input]” denotes subtracting q from all elements $\geq q/2$ in the input.

Number Theoretic Transform (NTT) (1/2)

- Generic Discrete Fourier Transform (DFT) equation:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot W_N^{kn}$$

- Commonly known DFT: (over complex field $\mathbb{C}$)
 - Operations occur on complex floating point values
 - Root of unity $W_N = e^{\frac{2\pi i}{N}} = \cos\left(\frac{2\pi kn}{N}\right) + i \cdot \sin\left(\frac{2\pi kn}{N}\right)$
- Fast Fourier Transform (FFT) = technique to apply the DFT in an efficient manner using “butterfly-like” combinations of values (from $O(n^2)$ to $O(n \log n)$)
- NTT = DFT over the ring $\mathbb{Z}/p$
 - Operations occur over integers (modulo p)
 - Root of unity $W_N = \text{some value } \alpha \text{ for which } \alpha^N \equiv 1 \pmod{p}$

- Since NTT has a similar structure to the commonly known DFT, FFT butterfly technique applies to the NTT as well

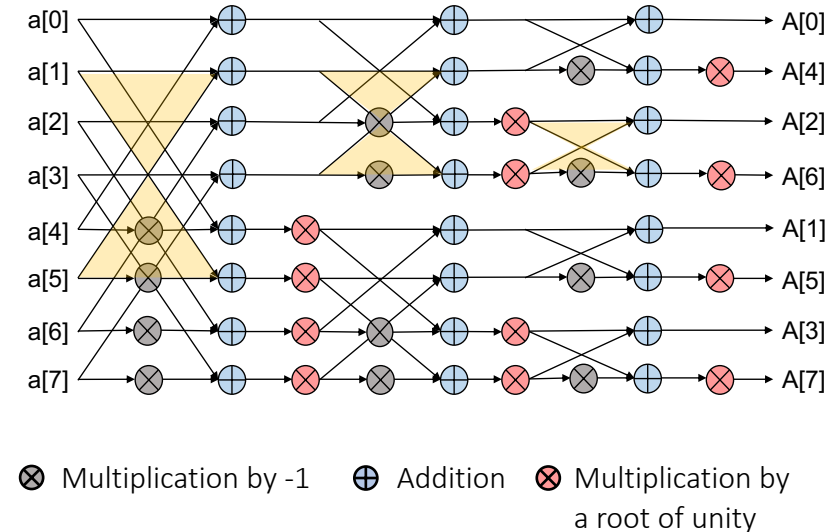


Figure 3. Visualization of an FFT algorithm. Yellow triangles show the “butterfly-like” combinations of values.

Number Theoretic Transform (NTT) (2/2)

- Application of NTT for polynomial multiplication of two size n polynomials a and b :

Regular polynomial multiplication

$$a * b = c$$


Regular multiplication ($O(n^2)$)

Polynomial multiplication using NTT

$$c = NTT^{-1}(NTT(a) \cdot NTT(b))$$


Pointwise multiplication ($O(n)$)

- Problem:** Multiplication of two polynomials of size n requires an output size of $2n - 1$. However, DFT-like algorithms operate on the same-sized polynomial throughout.

Solution 1: Zero-pad

- Requires input polynomials to contain at most $n/2$ coefficients, with the remaining coefficients set to zero
- We implement this as “NTT-1”

Solution 2: Use a “wrapped convolution” method

- For input polynomials in a ring, involves scaling the input polynomials by certain factors before applying transform
- Size n input polynomials will result in a size n output that represents the multiplication as it would occur in the ring
- We implement this as “NTT-2”

Empirical Analysis of LBC Data-Level Parallelism

- Our Goal: Investigate LBC schemes, identify bottlenecks, and exploit data-level parallelism
- Methodology
 - Measure runtimes of two LBC algorithms
 - Convert the algorithms to use SIMD-accelerated NTT kernel
 - Time all major operations of the algorithms
- After profiling the algorithms, we identify:
 - Using SIMD NTT reduces runtime by >98%
 - The component-wise workload performance breakdown, with remaining bottlenecks
 - Counting instructions suggests increasing SIMD widths helps, to a point

Runtime Experiments

- We utilized the Mayer-Toolkit², a C++ implementation of the CPKC and DSC LBC cryptosystems (Orig), with parameters and setup given in Figure 5, as a basis for our experiments
- First, we optimized the Mayer-Toolkit by rearranging some of the operations (Orig-R)
- Next, we implemented SIMD auto-vectorized regular (NTT-1) and negative-wrapped (NTT-2) NTT algorithms and measured speedup
- Results show our SIMD NTT implementations improved performance significantly
- Given their similar runtimes, NTT-2 algorithms are more efficient than NTT-1 algorithms for vectors of larger sizes

q	p	l	m
12289	3	2	2048

Figure 5. Parameters used for experiments. All experiments were performed on an Intel core i7 with icpc -O3 optimizations.

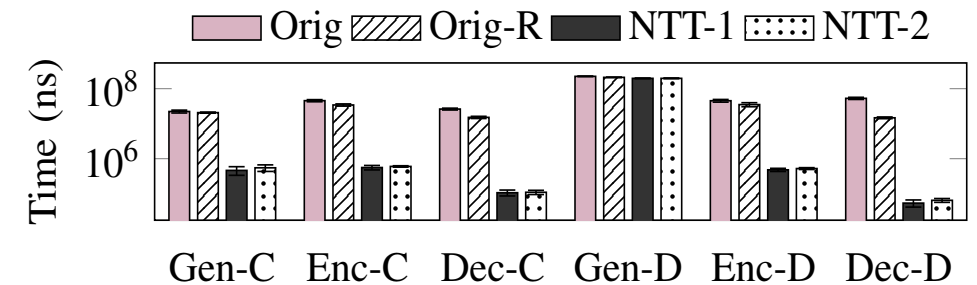


Figure 6. Full submodule runtimes for CPKC (C) and DSC (D) using the original (Orig) and reordered (Orig-R) Mayer-Toolkit operations and AVX2 autovectorized NTT algorithms (NTT-1 and NTT-2). “Gen” submodule consists of public key and secret key generation.

Workload Breakdowns

- In the original Mayer-Toolkit implementation (Orig), NTT consumed over >98% of the runtime
- Using the NTT-2 kernel, NTT no longer dominates
 - Encryption submodule now consists mainly of sampling from a discretized Gaussian distribution
 - Decryption submodule now consists equally of L_m/L_m^{-1} and $NTT_{m,q}^{-1}$ transformations
- Further analysis showed that the vector modulo operations consume >82% of the total instruction counts for the NTT algorithms
 - Efficient hardware implementation of modulus reduction would likely result in significant performance benefits

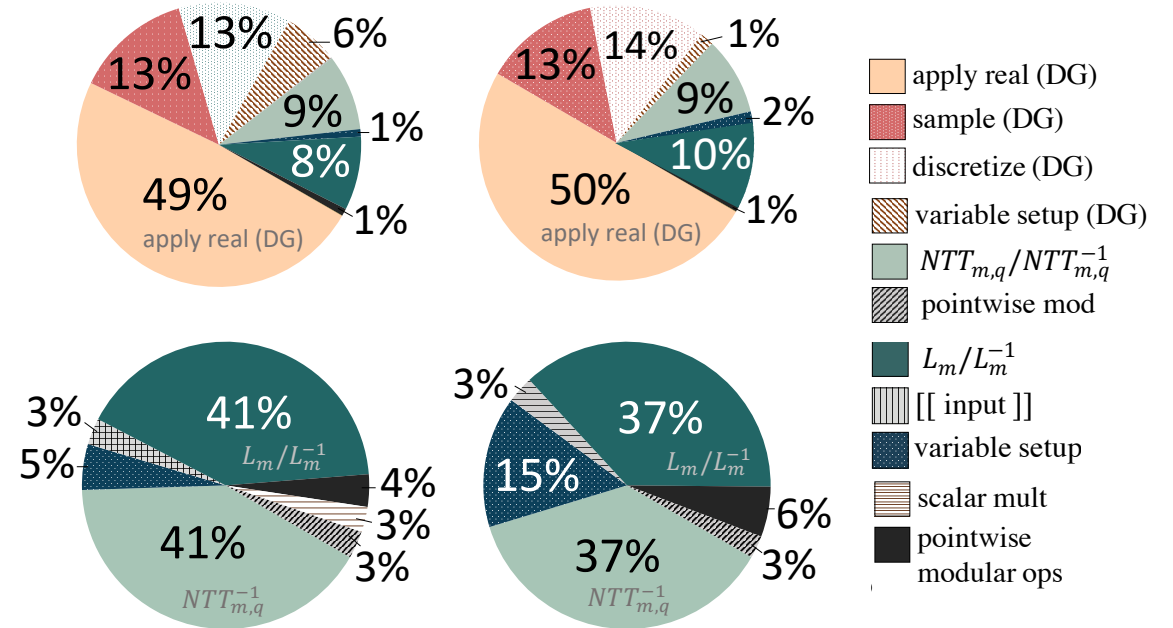


Figure 7. Workload breakdowns of CPKC (left) and DSC (right) Encrypt (top) and Decrypt (right) submodules using AVX2 autovectorized NTT-2 algorithms. Pointwise modulus operations include addition, multiplication, and negation. “DG” operations are components of the the discretized Gaussian sampling kernel.

Amenability to SIMD Vectorization

- Intel has been steadily increasing their processor SIMD register widths over time (see: Table 1)
- Given the increasing prevalence/growth of vectorization, how might LBC algorithms benefit as SIMD width continues to scale?
- To approximate potential performance improvements, we derived a parameterized instruction count of the NTT kernel and scaled as a function of future possible SIMD widths, treating modulo as one instruction to represent a system with efficient modulo extensions
- Results show initial improvement, followed by diminishing returns beyond a width of ~64

	MMX	SSE*	AVX	AVX512
Size (bit)	64-bit	128-bit	256-bit	512-bit
Size (32-bit int)	2	4	8	16
Year released	1997	1999	2011	2016

Table 1. Intel SIMD width increases over time.

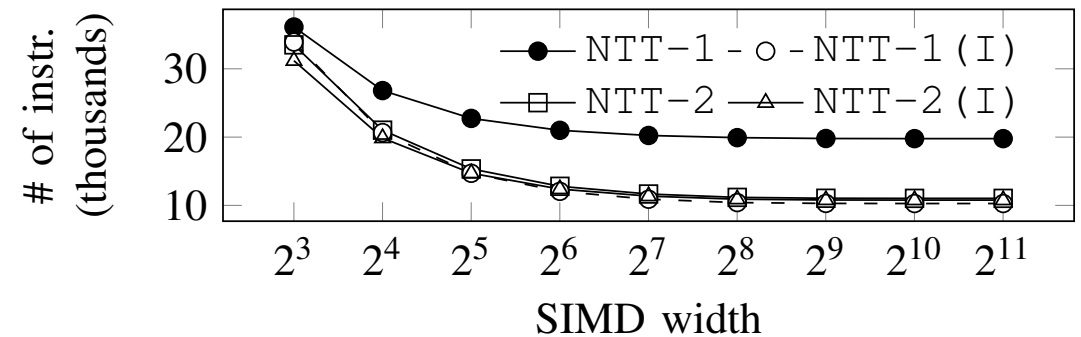


Figure 8. Number of instructions in the NTT kernels as a function of SIMD width. “I” refers to inverse NTT transformations. Instruction counts assume a single instruction per modulo operation, representing a system with efficient modulo extensions.

Discussion

- Many commonalities exist between LBC schemes → case for flexible lattice cryptographic accelerator
- LBC schemes can exhibit significant performance improvements using SIMD-autovectorized NTT techniques
- With NTT improvements, remaining bottlenecks are discretized Gaussian sampling for encryption and L_m/L_m^{-1} and inverse NTT for decryption
- Modulo reduction consumes a significant portion of vectorized NTT operations → opportunity for significant speedup with modulo acceleration
- Although increasing SIMD widths improve NTT kernel performance, only marginal improvements seen beyond a SIMD width of ~64